

Title	AXI DMA Controller Design Note
Project	None
Description	AXI DMA Controller Design Note
Author	holelee
Date	20/Jun/2005

(*) DMA Controller

기본적인 SoC 설계에 있어서 빠지지 않는 것이 DMA Controller이다. DMA Controller는 CPU의 관여 없이 특정 address에 있는 data를 다른 address로 옮기는 device를 말한다. 일반적인 주변장치에 input stream이나 output stream을 memory에 읽고 쓰는데 사용한다. bandwidth를 많이 점유하거나 latency가 중요한 주변장치의 경우에는 DMA Controller를 사용하지 않고 직접 BUS master가 되어 특정 address에서 data를 읽고 쓰도록 하는 경우가 많다. DMA Controller를 사용하게 되면 항상 read/write가 쌍으로 발생하게 되므로 필요한 BUS 점유시간이 2배로 늘어나게 된다. BUS가 cross-bar switching형식이라면 Memory로 접근하는 bandwidth에는 변화가 없지만, AHB와 같은 shared mux구조라면 BUS의 bandwidth를 2배 잡아먹게 된다. latency 측면에서는 DMA controller를 사용하게 되면 DMA request의 arbitration에 delay가 생기고 bus를 acquire하는 데 걸리는 시간이 두번 소요되므로 BUS master로 동작하는 경우에 비해서 data를 읽고 쓰는 latency가 증가하게 된다. 하지만 모든 주변 장치가 BUS의 master로 설계되면 BUS 자체의 clock을 높이는데 한계가 생기게 되고, 주변 장치를 BUS master로 설계하는 것 자체가 상당한 복잡도를 가지게 되므로 많은 bandwidth를 요구하지 않고, latency가 아주 중요하지 않는 느린 주변 장치의 경우에는 DMA Controller를 사용하게 된다.

(*) ARM PrimeCell DMA Controller

ARM에서 PrimeCell로 제공하는 DMA Controller는 PL080, PL081 2종류가 있는데 모두 AHB 기반이다.

- PL080

PL080은 8개의 channel을 가지고 16개의 DMA request를 지원하는 DMA controller이다. 독특한 부분은 각 channel별로 4 word FIFO를 가지고 있다는 점이다. 각 channel별로 FIFO를 가지도록 한 이유는 기본적으로 peripheral-to-peripheral transfer를 완벽하게 지원하기 때문인데, source peripheral의 burst size와 destination peripheral의 burst size가 다른 경우도 고려했기 때문이다. 즉 source에서 읽어 FIFO에 저장한 data를 모두 destination으로 전송하지 않고 destination이 필요한 만큼만 전송하기 때문에 DMA request를 처리하고도 FIFO에 data가 남을 수 있다.

PL080의 또 다른 특징은 AHB Master interface가 2개 있다는 점이다. 이 것은 아마도 Peripheral BUS와 Memory BUS를 구분하여 사용할 수 있도록 하기 위해서 인 것으로 보인다. Peripheral BUS와 Memory BUS를 구분하여 사용하게 되면 DMA를 통해 data를 전송할 때

memory bandwidth를 필요한 만큼만 사용할 수 있기 때문이다. 각 DMA Channel의 source와 destination이 어떤 AHB Master Interface를 사용하는 지 결정할 수 있는 register가 존재한다. 일반적인 DMA Controller에 없는 또 다른 특징은 Peripheral의 DMA request 신호가 BREQ(Burst Request), SREQ(Single Request), LBREQ>Last burst request), LSREQ>Last single request)로 모두 4개라는 점이다. 이것도 peripheral-to-peripheral 전송을 효율적으로 처리하기 위해서 존재하는 것인데, DMA를 통해서 전송할 data를 software programmer가 정확하게 알지 못하는 상황에 대비해 peripheral이 single request를 줌으로써 burst 단위가 아니라 정확한 size만큼 전송을 할 수 있다. 또한 last request라는 것을 peripheral이 알려줄 수 있도록 하여 DMA Controller가 전체 data 전송이 끝난 시점을 detect하여 interrupt 발생을 시킬 수 있도록 하였다.

PL080은 또한 Linked List data 구조를 이용하여 scatter-gather DMA를 지원한다. Scatter-gather의 동작은 전체 block의 transfer가 끝나면 Linked List data를 memory에서 읽어 control register(source address/destination address 등)에 write하는 방식으로 동작한다.

DMA request에 대한 arbitration은 단순한 priority based 방식을 사용하고 있다.

Artisan TSMC 0.18 um 공정에서 166MHz로 합성할 때 약 9만 5천 Gate count를 가진다.

- PL081

PL081은 PL080의 subset version으로 2 channel(16 DMA request)만 지원하고 AHB Master Interface가 하나만 존재한다.

Artisan TSMC 0.18 um 공정에서 166MHz로 합성할 때 약 2만 7천 Gate count를 가진다.

(*) Intel XScale(PXA255) DMA Controller

XScale에 있는 DMA Controller는 XScale의 전체 block diagram에 나타나 있듯이 peripheral bus(APB 추정)와 internal bus(AHB 추정) 사이에 bridge와 함께 존재한다. 위에서 언급한 memory bandwidth를 낭비하지 않으려는 의도로 보이는 데 이 점은 Samsung S3C2440도 동일하다.

기본적으로 memory-to-memory, memory-to-peripheral, peripheral-to-memory transfer를 지원하고, 40개의 request signal을 지원하는 16 channel DMA controller이다. PL080과 마찬가지로 descriptor based scatter-gather DMA를 지원한다. 그러나 각 channel 별로 FIFO가 존재하는 것 같지는 않다.

DMA request arbitration은 4개의 request를 묶어 4개의 group를 만든 후에 그것을 각 group 내에서는 round-robin으로 처리하고 group 사이에는 priority based로 처리하도록 되어 있다. Lowest priority를 가지는 group의 starvation을 막기 위해서 가장 높은 group이 4/8, 두 번째 group이 2/8, lowest group 두 개가 1/8씩의 배분을 받도록 되어 있다.

(*) AXI용 DMA Controller에 대한 생각

일반적인 MCU에서는 PL080과 같이 각 channel별로 별도의 FIFO를 가지고 있는 DMA Controller는 필요가 없다. AXI는 Cross-bar switching BUS를 기본으로 가정하고 있다. 따라서 특별히 peripheral bus와 memory bus를 구분할 필요는 없고, 또한 AHB와 달리 Write transfer와 read transfer를 동시에 수행할 수 있으므로 그것을 이용하면 더 좋은 성능을 가질 수 있다.

Read/write를 동시에 수행할 수 있으므로 두 개의 channel을 동시에 처리할 수 있다. 즉 한 channel은 read를 수행하고, 다른 channel은 이미 read하여 FIFO에 저장되어 있는 data를 write할 수 있다. 따라서 FIFO가 두개 필요하고 AXI의 burst size는 16까지이므로 16 word의 FIFO를 가지면 된다.

또 고려해야할 점으로, 현재 설계되고 있는 AXI SDRAM Controller가 byte/halfword에 대해서는 burst를 지원하지 않으므로 byte/halfword를 지원하는 peripheral의 경우에 대해서는 FIFO에서 word로 packing하여 transfer를 해야 한다.

(*) GDMA

탐내에서 이미 가지고 있는 DMA Controller인 GDMA는 4 channel를 지원하는 DMA controller이다. 기본적인 기능만을 수행하고 FIFO에 pack/unpack을 수행하지도 않아 직접 사용하기에는 무리가 있다. 4 channel DMA의 합성 결과 4만 gate 정도 되므로 PL080에 비해서도 그렇게 작다고 할 수 없다.

(*) PL080의 사용

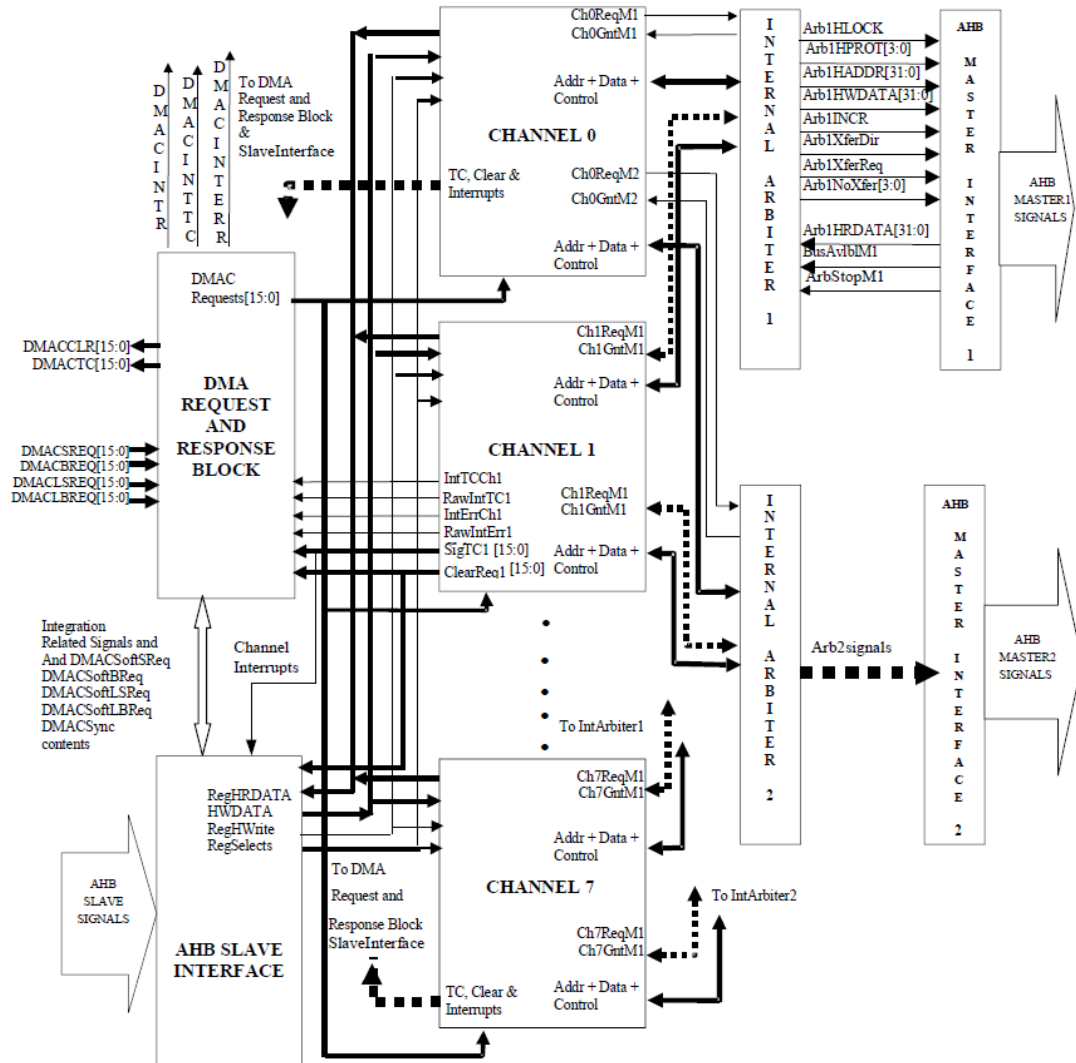
PL080을 AXI에서 사용하기 위해서는 현재 설계되어 있는 AHB master to AXI bridge를 연결하고, PL080의 register programming interface인 AHB Slave interface를 APB interface로 변경하면 된다. 또한 두 개의 AHB master interface를 가지도록 만들 이유는 없으므로 하나의 AHB master interface를 제거해야 한다.

이런 간단한 수정만으로는 다음과 같은 몇가지 문제가 있다.

- hardware size가 너무 크다
- AXI의 강점을 활용할 수 없다.
- PrimeCell을 editing하므로 license문제가 있을 수 있다.
- 상대적으로 작은 FIFO의 크기

Hardware size가 큰 이유는 PL080의 구조와 관련이 있다. PL080은 다음 그림과 같은 구조로 되어 있다. 기본적으로 각 channel별로 Register와 FIFO, State machine들이 들어 있는 channel processing unit이 있고, 그것을 8개 사용하고 있다. DMA request를 arbitration 하는 구조가 아니라 각 channel processing unit이 출력하는 bus request를 arbitration하여 AHB Master interface로 출력하는 구조로 되어 있다. 이런 구조는 단일 channel만 고려하면 되므로 design이 간단하다. 또한, channel 수를 늘이거나 줄일 때, 단순히 channel processing unit의

instance 갯수를 조정하고 BUS arbitration을 바꾸면 된다. 그리고 각 channel이 독립적으로 DMA request를 처리하므로 효율적일 가능성이 있다. 하지만 AHB는 기본적으로 wait구조를 가지므로 각 channel별로 DMA request를 독립적으로 처리하더라도 AHB BUS에서는 data가 순차적으로 전송되게 되므로 큰 효율성을 발휘하기 힘들다.



AHB에 비해 AXI가 지니는 강점은 ID를 이용한 out-of-order completion과 read와 write를 동시에 처리할 수 있다는 점이다. 우선 out-of-order completion을 이용하여 성능이 향상될 가능성이 있는 부분은 각 channel의 source에서 read가 일어날 때이다. destination에 write할 때는 write data interleaving을 사용하지 않는 이상, data 전송이 순차적으로 일어날 수 밖에 없으므로 큰 성능 향상을 얻기 힘들다. read에 대해서 성능을 향상시키기 위해서는 PL080과 같이 arbitration을 channel이 Bus request를 하는 부분에서 수행하도록 한다. 그 다음 각 channel별로 서로 다른 ID로 read request를 전송하고 out-of-order로 돌아오는 read data를 ID로 구분하여 각 channel에 전달하면 된다. Read request를 빨리 전송할 수 있고 서로 다른

slave에서 read가 동시에 수행될 가능성이 있으므로 성능 향상이 있을 수 있다.

Read/Write를 동시에 처리할 수 있는 AXI interface의 장점을 활용하기 위해서는 PL080의 구조에서는 Bus Arbiter 2개를 한쪽은 read request만, 다른 한쪽은 write request만을 처리하도록 조정한 다음에 처리하면 된다. PL080은 각 channel별로 source와 destination, linked list data에 대해서 어떤 AHB master interface를 사용할 지 결정할 수 있는 register bit가 존재하는데 이것을 상수로 하게 되면 쉽게 조정된다.

여기에는 한가지 문제가 있을 수 있는데 PL080의 각 channel processing unit의 종료시점이 일치했을 때, 다른 block과의 통신이 동시에 일어날 경우에 예러가 생길 가능성이 있다. 이 부분을 확실히 확인해야 한다.

PrimeCell의 수정 권한에 대한 문제는 나중에 생각하기로 한다.

각 채널에 존재하는 FIFO의 크기가 4 words 밖에 되지 않는다. 이는 곧 burst 8이나 burst 16 word transfer를 할 수 없다는 말이고, SDRAM을 접근할 때는 성능 저하가 일어날 가능성이 있다. SDRAM에 4 words를 모두 읽은 다음에 FIFO가 소진될 때까지 더 이상 SDRAM에 read request를 보낼 수 없다. 따라서 SDRAM의 latency에 영향을 받게 된다. FIFO의 크기를 늘리고 double buffering 등을 사용하면 좋겠지만 그렇게 수정하기에는 시간이 촉박하므로 우선 그냥 사용하기로 한다.

(*) 새로 Design하는 방법

새로 설계하는 방법은 두 가지 정도를 생각해 볼 수 있다. Hardware size에 최적화된 설계와 성능에 최적화된 설계 방법 이다. 우선 성능에 최적화된 설계는 기존의 PL080과 크게 다르지 않다. 각 channel별로 processing unit를 따로 구성하고 BUS request를 arbitration하는 방법 이다. 다만 PL080에 있는 feature중에 일반적인 사용에서는 사용하지 않는 peripheral-to-peripheral DMA에서 flow control을 last request 신호를 이용하여 peripheral이 직접 control하는 부분 등과 같이 복잡한 동작은 하지 않도록 만드는 것이다.

Hardware size에 최적화된 설계는 각 channel별로 processing unit을 두는 것이 아니라 processing unit이 하나 있고 각 channel별로 가지는 register set을 processing unit이 읽어와 처리하는 방법이다. 즉 BUS request를 arbitration하는 것이 아니라, 각 channel의 DMA request를 arbitration하여 한번에 하나씩 DMA request를 처리하는 방법이다. 이 경우에는 약간의 설계 복잡도가 증가하게 되지만, FIFO와 state machine을 하나만 사용할 수 있으므로 hardware size가 줄어든다. 하지만 어떤 시점에서 DMA request를 하나만 처리하게 되므로 전체적인 성능은 떨어질 것이다. Processing unit을 두 개나 그 이상 두어 한 번에 여러개의 DMA request가 처리될 수 있도록 할 수 있는데 그렇게 하면 AXI의 장점인 read/write를 동시 처리를 이용할 수 있다.

(*) Test 방법

PL080을 변경하든, 새로 설계하든 설계가 끝난 RTL code를 test해야 한다. Test를 진행하기

위해서 기본적으로 BUS가 필요하다. BUS에 기본적으로 TestMaster와 DMA Controller, Memory(Synchronous SRAM), test용 peripheral을 연결하기 위한 AXI2APB Bridge를 붙인 다음 테스트를 진행하게 된다. DMA Controller의 register setting은 TestMaster로 수행하게 되는데 TestMaster는 interrupt handling이 되지 않으므로 이를 고려해야 한다. Test용 peripheral은 두 종류로 만들어야 한다. DMA Controller를 이용하여 memory를 읽는 device와 DMA Controller를 이용하여 memory에 write하는 device이다. Random한 behavior를 가지는 것이 좋으므로 각 peripheral은 ACLK을 적당히 분주한 후, data를 read/write하는 부분을 random하게 처리하여 DMA request하는 시점이 서로 다르게 될 수 있도록 처리해야 한다.

- Test 시나리오

- TestMaster는 0~4KB의 memory 영역에 0~1023의 값을 써 넣어 둔다.
- DMA Controller를 사용하여 0~4KB의 memory영역을 다른 memory 영역에 복사하는 test를 진행한다.
- DMA Controller를 이용하여 peripheral과 memory간의 data transfer를 setting한다.
 - Peripheral-to-memory channel은 4KB~8KB, 8KB~12KB, 12KB~16KB의 영역에 write하도록 setting한다.
 - Memory-to-peripheral channel은 0~4KB의 memory 영역을 읽도록 한다.
 - Memory-to-Memory channel은 0~4KB의 memory를 읽어 0~4KB에 다시 쓰도록 한다.
 - 3개의 channel이 p-to-m/4개의 channel이 m-to-p/1개의 channel이 m-to-m을 담당하게 된다.
- 각 peripheral을 enable 시켜 DMA를 동작시킨다.
- DMA Controller부터 모든 channel에 대한 interrupt를 받으면 TestMaster는 Memory에 쓰여진 값을 모두 read하여 검사한다.

- test용 peripheral

- FIFO 사용 : Fixed address burst만 허용
- Word/Byte/Halfword 대응
- random한 timing을 가지고 FIFO를 채우거나 비움
- FIFO의 level에 따라 DMA request 생성
- M-to-P test를 위한 peripheral의 경우 FIFO에서 data를 읽어서 data의 정확성을 검사하게 된다.(0~1023)
- Programmable register
 - Enable/Disable bit
 - Finished bit(read only)

- Test platform 구축 과정

- PL080의 AHB Slave interface를 APB로 변경
- PL080의 AHB Master Interface 두 개에 AHB2AXI Bridge 연결
- BUS에 PL080,TestMaster,AXI2APB Bridge, Internal SRAM Controller 연결
- APB에 test peripheral 연결
- TestMaster Programming
- PL080의 AHB Master Interface 부분을 들어내고 AXI interface로 바꾼 후 다시 test

(*) 새로 DMA Controller를 설계할 경우

- 각 Channel별 register

(a) Descriptor Address : enable bit 포함

(b) Source Address

(c) Destination Address

(d) Control

source width(byte,halfword,word) : 2 bit

destination width(byte,halfword,word) : 2bit

fixed/increment setting : 1 bit for each

burst size(1~64) : byte

Transfer counter(4KB를 꼭 채울 정도, 크면 클 수록 좋음) : 13 bit 이상

memory-to-memory transfer 여부(Req 신호 무시) : 1 bit

(e) control & status

enable/disable bit : 1 bit

Interrupt enable/disable bit(Transfer counter가 0이 됨, Descriptor가 load됨, Descriptor가 끝남, Error detect상황) : 4 bit

active Read Only register : 1 bit

interrupt status

interrupt ack

request pended bit

- Other registers

Channel별로 관리되는 register를 제외하고 전체를 control하는 register로는 다음과 같은 것이 필요하다.

Interrupt for each channel

Interrupt clear register

DMAReq mux 관련

enabled channel read only register

- Channel의 FIFO depth

AXI는 maximum 16 burst를 지원하므로 depth 16인 FIFO를 사용하기로 한다.

- Channel별 input/output signal

DMAReq/DMAAck/Interrupt관련/Memory Request관련

- 전체적인 구조

(a) 가장 빠른 구조

다음과 같은 구조를 가지는 것이 DMA controller가 가장 빠른 속도로 동작할 가능성이 있다.

한 channel을 처리할 수 있는 DMA Controller를 AXI Master로 하나씩 연결한 형태를 가진다. 이 방법은 DMA arbitration이 모두 BUS에서 처리되게 된다. 이 방법의 단점은 우선 Hardware size가 크다는 것이고, BUS에 너무 많은 AXI Master가 추가됨으로 인해 전체적인 BUS clock에 제한을 받을 수 있다는 점이다.

(b) 가장 빠른 구조 변형

위에서 언급한 가장 빠른 구조가 가지는 단점 하나를 보완한 것이 다음과 같은 구조이다. 각 channel의 AXI Master Interface에 arbiter를 달아 하나의 AXI Master Interface를 가지도록 변경한 것이다. 이 경우 system bus에 단일 master로 물리게 되므로 system BUS clock의 속도에는 큰 문제를 일으키지 않는다. 또한 arbiter에서 DMA Request arbitration을 처리할 수 있으므로 원하는 arbitration scheme을 구현하면 된다. Priority based arbitration을 한다면 arbiter를 별도로 설계하지 않고, N개의 master와 1개의 slave를 가지는 AXI BUS를 구성하여 연결해도 무방하다.

(c) 가장 작은 구조

다음과 같은 구조가 가장 작은 구조를 가지는 DMA Controller이다. 이 것은 각 channel별로 DMA request를 처리하는 것이 아니라 DMA request를 처리할 수 있는 engine을 하나 두고 각 channel이 time sharing하여 사용하는 방법이다. DMA request의 arbitration은 말 그대로 request를 arbitration하여 하나의 channel을 선택하게 된다. 그 channel의 burst transfer(read/write)가 모두 끝나면 arbitration을 새로 수행하여 다음 request를 찾아 수행하는 구조를 가진다. 이 구조의 문제는 read와 write가 순차적으로 일어날 수 밖에 없다는 점이다. AXI의 장점인 read/write가 동시 수행이나, out-of-order completion을 통한 성능 향상을

기대할 수 없다는 점이다. AHB와 같은 BUS에서는 이런 구조를 가지는 것이 가장 좋겠으나 AXI에서는 성능면에서 좋은 구조는 아니다.

(d) 절충

이 구조는 가장 작은 구조의 DMA Controller를 여러개 만들고 그것을 다시 arbitration한 것이다. 이 구조는 어느 정도의 절충이라고 볼 수 있다.

- DMA Engine

DMA Engine은 다음과 같은 입력을 받는다.

(a) Channel Register 내용

(b) Start signal

(c) Memory Interface

DMA Engine의 출력은 다음과 같다.

(a) Channel register중 update될 내용

(b) Memory Interface

(c) Done signal

(d) Interrupt signal

- Byte/Halfword 단위 access 문제

현재 설계된 SDRAM Controller는 Byte/Halfword read/write가 burst로 처리되지 못한다. Word transfer는 burst로 수행할 수 있지만 Byte/Halfword의 read/write는 반드시 single로 처리해야 한다. 그런데 기본적으로 DMA를 수행하는 device가 word 단위의 data를 공급하지 못하고 byte나 halfword 단위의 data를 공급할 가능성이 크다는 점이다. 예를 들어 USB device는 data를 byte단위로 처리한다. 따라서 byte와 halfword data를 word data로 packing/unpacking을 수행할 수 있어야 한다.

(a) unaligned SDRAM address

Peripheral에서 byte data stream을 SDRAM으로 전송할 때 초기 SDRAM address가 word align이 맞지 않은 경우를 생각해 볼 수 있다. 물론 user에게 SDRAM address는 항상 word aligned로 맞추라고 강요할 수도 있는 법이지만 user가 그렇게 사용할 수 없는 경우가 존재한다. 그렇다면 unaligned SDRAM address에 대해서 처리를 수행해야 하는데 그 때는 다음과 같이 처리한다.

(b) trailing bytes

한번의 DMA request마다 전송되는 byte의 숫자는 반드시 4의 배수가 되도록 맞추도록 user에게 강제해도 무방하다. FIFO의 depth를 늘리면 그만이기 때문이다. 그러나 전체 전송 byte의 갯수는 peripheral 마다 4의 배수가 안되는 경우가 존재한다. 예를 들어 USB device로 전송되

는 data는 꼭 4의 배수인 byte 갯수를 가지지 않을 수 있다.